분석기 골탕 합성 문제 구체화

정원준

2025년 11월 7일

ROPAS@SNU

목차

문제 정의

간단히 설명하면

```
x = readInt
if x * x - 100 * x + 2500 == 0:
    x = 0
print x
```

간단히 설명하면

마지막에서 x의 값은?

```
x = readInt
if x * x - 100 * x + 2500 == 0:
    x = 0
print x
```

간단히 설명하면

```
x = readInt
if x * x - 100 * x + 2500 == 0:
    x = 0
print x
```

마지막에서 x의 값은?

나: $x \in \mathbb{Z} \setminus \{50\}$

간단히 설명하면

```
x = readInt
if x * x - 100 * x + 2500 == 0:
    x = 0
print x
```

마지막에서 x의 값은?

나: $x \in \mathbb{Z} \setminus \{50\}$

분석기: $x \in \mathbb{Z}\ (= \top)$

간단히 설명하면

```
x = readInt
if x * x - 100 * x + 2500 == 0:
    x = 0
print x
```

마지막에서 x의 값은?

나: $x \in \mathbb{Z} \setminus \{50\}$

분석기: $x \in \mathbb{Z}\ (= \top)$

좀 더 좋은 분석기:

$x \in \mathbb{Z} \setminus \{50\}$

Rice 정리

Rice 정리

Theorem (Rice's Theorem, Rice 1951)

φ를 부분 계산 가능한 함수들의 허용 가능한 넘버링*admissible numbering* 이라고 하자. $P \subseteq \mathbb{N}$을 어떤 성질을 만족하는 프로그램들의 인덱스 집합이라고 하자. 다음 두 조건을 만족한다고 가정하자:

1. P는 비자명(***non-trivial***)하다. 즉, $P \neq \emptyset$ 이고 $P \neq \mathbb{N}$이다.
2. P는 확장적(***extensional***)이다. 즉, 모든 $m, n \in \mathbb{N}$에 대해 $\varphi_m = \varphi_n$이면 $m \in P \iff n \in P$이다.

그렇다면, P는 결정 불가능(***undecidable***)하다.

Rice 정리

Theorem (Rice's Theorem, Rice 1951)

φ를 부분 계산 가능한 함수들의 허용 가능한 넘버링*admissible numbering* 이라고 하자. $P \subseteq \mathbb{N}$을 어떤 성질을 만족하는 프로그램들의 인덱스 집합이라고 하자. 다음 두 조건을 만족한다고 가정하자:

1. P는 비자명(***non-trivial***)하다. 즉, $P \neq \emptyset$ 이고 $P \neq \mathbb{N}$이다.
2. P는 확장적(***extensional***)이다. 즉, 모든 $m, n \in \mathbb{N}$에 대해 $\varphi_m = \varphi_n$이면 $m \in P \iff n \in P$이다.

그렇다면, P는 결정 불가능(***undecidable***)하다.

"뭐라고요?"

우리 상황에 맞게

이 정리를 우리 문제 상황에 맞게 가공하면 다음과 같다:

우리 상황에 맞게

이 정리를 우리 문제 상황에 맞게 가공하면 다음과 같다:

Corollary

$\mathbb{L}$을 튜링 완전한 프로그래밍 언어라고 하자. $\mathscr{P} \subseteq \mathbb{L}$을 프로그램의 부분집합, 혹은 성질이라 하자. 다음 두 조건을 만족한다고 가정하면:

1. $\mathscr{P}$는 **의미 있는 성질**이다. 즉, $\mathscr{P} \neq \emptyset$ 이고 $\mathscr{P} \neq \mathbb{L}$이다.
2. $\mathscr{P}$는 **실행의미에 딱맞는 성질**이다. 즉, $\forall p_1, p_2 \in \mathbb{L}$에 대해 p_1과 p_2의 실행의미가 같으면 $p_1 \in \mathscr{P} \iff p_2 \in \mathscr{P}$이다.

다음과 같은 (항상 유한 시간 안에 끝나는)분석기 $\mathscr{A}$는 **존재하지 않는다**:

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \iff p \in \mathscr{P}$$

다른 말로, 안전$_{sound}$하면서 완전$_{complete}$한 분석기를 만들 수 없다.

공격 목표

안전$_{sound}$한 분석기 $\mathscr{A}$를 가정 즉,

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \Rightarrow p \in \mathscr{P}$$

이다.

공격 목표

안전$_{sound}$한 분석기 $\mathscr{A}$를 가정 즉,

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \Rightarrow p \in \mathscr{P}$$

이다. 그런데 Rice 정리에 의해

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \iff p \in \mathscr{P}$$

일 수 없으므로

공격 목표

안전$_{sound}$한 분석기 $\mathscr{A}$를 가정 즉,

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \Rightarrow p \in \mathscr{P}$$

이다. 그런데 Rice 정리에 의해

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \iff p \in \mathscr{P}$$

일 수 없으므로

$$\forall p \in \mathbb{L}, \mathscr{A}(p) = true \Leftarrow p \in \mathscr{P}$$

가 거짓이다.

공격 목표

$\mathscr{P}$를 "프로그램 종료 후 변수 x 가 가질 수 있는 값은 $\mathbb{Z}$ 전체가 아니다."
로 두면, 다음과 같은 명제를 도출할 수 있다:

$\exists p \in \mathbb{L}$,

$$\mathscr{A}(p) = false \text{ 그리고}$$

프로그램 종료 후 변수 x 가 가질 수 있는 값은 $\mathbb{Z}$ 전체가 아니다.

공격 목표

$\mathscr{P}$를 "프로그램 종료 후 변수 x 가 가질 수 있는 값은 $\mathbb{Z}$ 전체가 아니다."
로 두면, 다음과 같은 명제를 도출할 수 있다:

$\exists p \in \mathbb{L}$,

$$\mathscr{A}(p) = false \text{ 그리고}$$

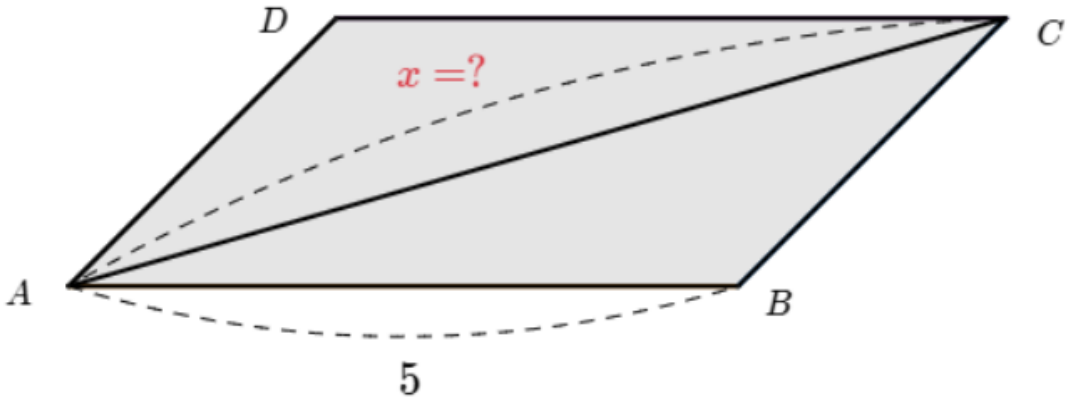
프로그램 종료 후 변수 x 가 가질 수 있는 값은 $\mathbb{Z}$ 전체가 아니다.

이를 만족하는 p를 찾는 것이 목표이다.

가능성

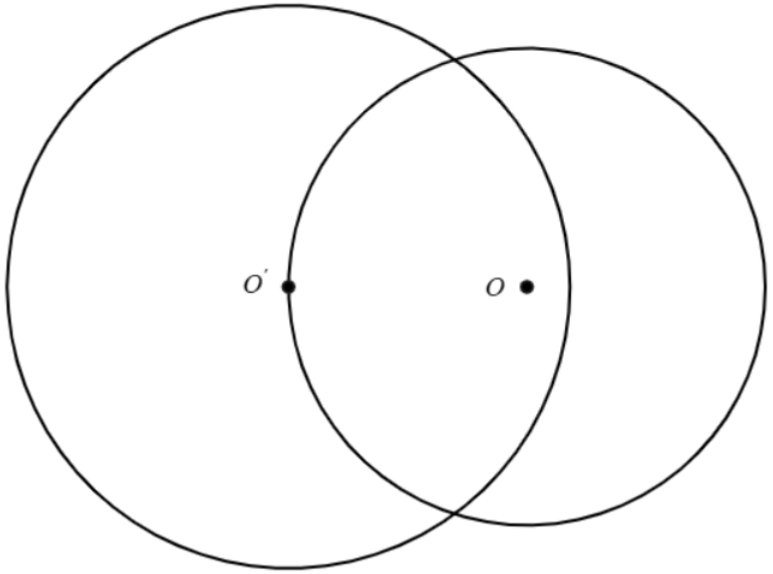
웬 가능성?

평행사변형 $ABCD$가 있다. 밑변 AB의 길이가 5이고 넓이가 10일 때, 대각선 AC의 길이는 얼마인가?



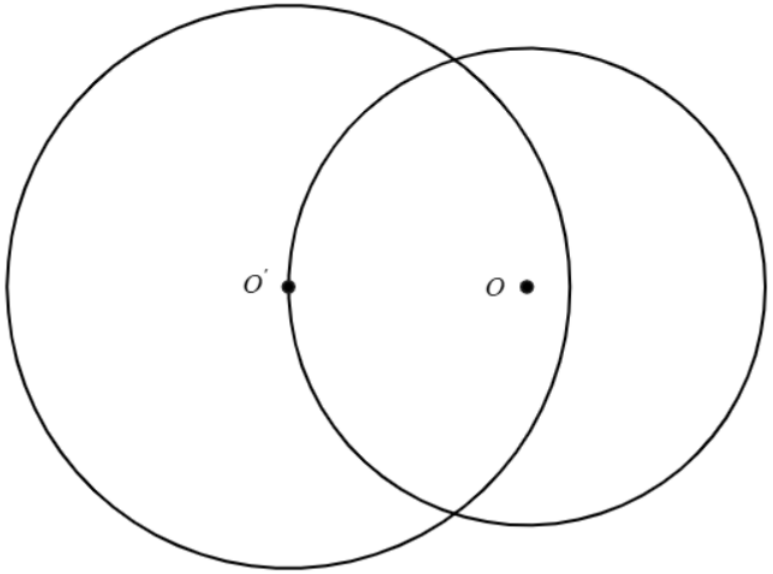
웬 가능성?

반지름의 길이가 1인 원 O와 O의 한 점을 중심으로 하는 원 O'가 있다.
O'가 O의 넓이를 2등분할 때, O'의 반지름의 길이는 얼마인가?



웬 가능성?

반지름의 길이가 1인 원 O와 O의 한 점을 중심으로 하는 원 O'가 있다.
O'가 O의 넓이를 2등분할 때, O'의 반지름의 길이는 얼마인가?



연구에 대한 정적 분석

전탐색

recall Rice 정리, 답을 만족하는 프로그램 p가 어딘가엔 존재한다.

전탐색

recall Rice 정리, 답을 만족하는 프로그램 p가 어딘가엔 존재한다.

그런데 그 p를 실제로 유한 시간 안에 찾는 알고리즘이 존재하는지는 모른다.

전탐색

recall Rice 정리, 답을 만족하는 프로그램 p가 어딘가엔 존재한다.

그런데 그 p를 실제로 유한 시간 안에 찾는 알고리즘이 존재하는지는 모른다.

프로그램의 개수는 셀 수 있으니까, 가능한 모든 p를 빠짐없이 하나하나 따져나가면 가능하지 않을까?

전탐색

recall Rice 정리, 답을 만족하는 프로그램 p가 어딘가엔 존재한다.

그런데 그 p를 실제로 유한 시간 안에 찾는 알고리즘이 존재하는지는 모른다.

프로그램의 개수는 셀 수 있으니까, 가능한 모든 p를 빠짐없이 하나하나 따져나가면 가능하지 않을까?

안 됨. 그렇게 찾은 i번째 프로그램 p_i가 실제로 $p_i \in \mathscr{P}$인지 알아내는 알고리즘이 없음(다시 Rice정리).

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

세상에 참인 명제는 셀 수 있이 많은데, 분석기가 그 중 하나쯤은 모르지 않을까?

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

세상에 참인 명제는 셀 수 있이 많은데, 분석기가 그 중 하나쯤은 모르지 않을까?

분석기는 공격기를 볼 수 없지만, 분석기는 공격기를 볼 수 있다.

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

세상에 참인 명제는 셀 수 있이 많은데, 분석기가 그 중 하나쯤은 모르지 않을까?

분석기는 공격기를 볼 수 없지만, 분석기는 공격기를 볼 수 있다.

힐베르트 프로그램을 흉내 내어, 참인 명제를 순서대로 마구 생성해가면서 분석기에게 이를 시험해 보면 되지 않을까?

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

세상에 참인 명제는 셀 수 있이 많은데, 분석기가 그 중 하나쯤은 모르지 않을까?

분석기는 공격기를 볼 수 없지만, 분석기는 공격기를 볼 수 있다.

힐베르트 프로그램을 흉내 내어, 참인 명제를 순서대로 마구 생성해가면서 분석기에게 이를 시험해 보면 되지 않을까?

분석기가 무한히 이어지는 모든 시험에 통과하기는 정말 어려울 것 같다.

참인 명제 사용

처음에 보여줬던 예시: "$x = 50$일 때, $(x^2 - 100x + 2500) = 0$이다."
라는 명제 사용.

세상에 참인 명제는 셀 수 있이 많은데, 분석기가 그 중 하나쯤은 모르지 않을까?

분석기는 공격기를 볼 수 없지만, 분석기는 공격기를 볼 수 있다.

힐베르트 프로그램을 흉내 내어, 참인 명제를 순서대로 마구 생성해가면서 분석기에게 이를 시험해 보면 되지 않을까?

분석기가 무한히 이어지는 모든 시험에 통과하기는 정말 어려울 것 같다.

정말 이렇게 해보겠다는 건 아니고, 만약 이 방법이 정말 어렵지만 성공적이라면, 합성으로 우회해서도 가능하지 않을까?

G 언어

문법

$$
\begin{array}{lcl}
P & \rightarrow & C \\
\\
C & \rightarrow & x := E \\
 & | & C; C \\
 & | & \text{ifp } E\ C\ C \\
 & | & \text{whilep } E\ C \\
\\
E & \rightarrow & n \quad (n \in \mathbb{Z}) \\
 & | & x \\
 & | & E{+}E \\
 & | & E{*}E \\
 & | & {-}E
\end{array}
$$

실행의미

무엇 의미구조*denotational semantics*로 정의한다.

$$
\begin{aligned}
Val &= \mathbb{Z} \\
n &\in Val \\
M &\in Mem = Var \xrightarrow{\text{fin}} Val \\
x &\in Var \\
P, C &\in Cmd \\
E &\in Exp \\
[\![\cdot]\!]_{(C)} &: Cmd \to Mem \to Mem \\
[\![\cdot]\!]_{(E)} &: Exp \to Mem \to Val
\end{aligned}
$$

실행의미

$[\![E]\!]_{(E)}$ 의 의미

$$
\begin{aligned}
[\![n]\!]M &= n \\
[\![x]\!]M &= M(x) \\
[\![E_1 + E_2]\!]M &= [\![E_1]\!]M + [\![E_2]\!]M \\
[\![E_1 * E_2]\!]M &= [\![E_1]\!]M \times [\![E_2]\!]M \\
[\![-E]\!]M &= -[\![E]\!]M
\end{aligned}
$$

$[\![C]\!]_{(C)}$ 의 의미

$$
\begin{aligned}
[\![x := E]\!]M &= M\{x \mapsto [\![E]\!]M\} \\
[\![C_1; C_2]\!]M &= [\![C_2]\!]([\![C_1]\!]M) \\
[\![\text{ifp } E\ C_1\ C_2]\!]M &= if\ [\![E]\!]M > 0\ then\ [\![C_1]\!]M\ else\ [\![C_2]\!]M \\
[\![\text{whilep } E\ C]\!]M &= lfp\ (\lambda f.\lambda M.if\ [\![E]\!]M > 0\ then\ f([\![C]\!]M)\ else\ M)
\end{aligned}
$$

참고사항

참고사항

- G 언어의 문법에는 입/출력이 없다.

참고사항

- G 언어의 문법에는 입/출력이 없다.
- 대신 프로그램 전체의 실행의미를 $[\![P]\!]_{(C)}$ 로 정의하여 초기 메모리의 상태를 입력, 나중 메모리의 상태를 출력으로 생각한다.

참고사항

- G 언어의 문법에는 입/출력이 없다.
- 대신 프로그램 전체의 실행의미를 $[\![P]\!]_{(C)}$ 로 정의하여 초기 메모리의 상태를 입력, 나중 메모리의 상태를 출력으로 생각한다.
- non-relational 요약해석으로는 모든 변수가 $\top$ 인 상태로 시작한다. (프로그램이 무한루프에 빠지는지, 도달하지 못하는 라인이 있는지에는 관심 없다)

합성 방법

$\top$이 아님을 어떻게 보장?

- 앞서 언급했듯이, 아무렇게나 p를 만들어 놓으면 진짜로 $p \in \mathscr{P}$ 인지 알기 어려움

$\top$이 아님을 어떻게 보장?

- 앞서 언급했듯이, 아무렇게나 p를 만들어 놓으면 진짜로 $p \in \mathscr{P}$ 인지 알기 어려움
- 따라서 빈 칸이 있는 프로그램에 대해 이미 알고 있는 정보를 들고다니면서 합성해나가야 할 것

$\top$이 아님을 어떻게 보장?

- 앞서 언급했듯이, 아무렇게나 p를 만들어 놓으면 진짜로 $p \in \mathscr{P}$ 인지 알기 어려움
- 따라서 빈 칸이 있는 프로그램에 대해 이미 알고 있는 정보를 들고다니면서 합성해나가야 할 것
- 아직 잘 모르겠음

마저할일

마저할일

- 간단한 분석기 $\mathscr{A}$ 구현하기
- 마구 합성 시도해보기
- 잘 안 될 때마다 왜 안 되는지 명료하게 기록하기

감사합니다